Programación 4

Guía práctica del recuperatorio

Preparación para el lunes 26 de octubre de 2026. El objetivo es leer una consigna, identificar qué estado cambia y escribir las pocas líneas que resuelven el problema. Incluye soluciones explicadas de los modelos que compartiste; no incluye un simulacro para responder ahora.

1. Qué estudiar primero

La información sobre el recuperatorio es cambiante. Estas prioridades se basan en lo que relataste del profesor y en los ejercicios, no en una garantía sobre el examen.

Prioridad	Qué dominar	Qué deberías poder resolver
Central	useState, onChange y variables	Declarar un estado, leer el input y actualizarlo.
Central	Condiciones y disabled	Permitir buscar desde 3 caracteres e impedir valores fuera de un límite.
Central	useEffect y orden del componente	Distinguir carga inicial de carga por selección; ordenar las cajas.
De respaldo	Spread, handleChange, && y ternario	Actualizar un campo de un objeto sin borrar los demás; mostrar o bloquear según una condición.
Secundaria	Hook useLocalidades y filtrado con efecto	Entender el contrato del hook y cómo sincronizar una lista filtrada.

Los nombres que anotaste

- “Sprice Operator” probablemente significa **spread operator**, escrito `....`
- “Handell Change” probablemente significa **handleChange**, un nombre habitual para el manejador de cambios. No es una función incorporada de React.
- “Appersen Appersen” probablemente significa **ampersand ampersand**, escrito `&&`.
- `onChange` es el evento del input; `handleChange` puede ser la función que le pasás. `disabled` es una propiedad booleana del botón, no un evento.

Enfoque para este examen

Por lo que contás, el profesor prioriza el manejo de variables y el orden del código. La guía empieza por eso y deja Context como respaldo. No necesitás aprender a construir un proveedor de Context para consumir un hook ya provisto.

La frase que recordás como `onChange{Disable = {Buscar.Leghnt > 3}}` no es JSX válido. No podemos saber si fue una explicación abreviada o una transcripción inexacta. Aprendé la versión correcta: `disabled={buscar.length < 3}`. Los capítulos 4 y 5 explican el límite y el lugar de cada propiedad.

Índice

Hacé clic en un tema o utilizá los marcadores del PDF. Los capítulos 2 a 10 forman el núcleo; 11 y 12 son respaldo.

1. Qué estudiar primero 1

2. Estado y manejo de variables 3

3. Input controlado y eventos 4

4. Búsqueda con al menos 3 caracteres 5

5. Condiciones y símbolos que pueden aparecer 6

6. Spread y handleChange con objetos 7

7. useEffect y sus dependencias 8

8. Ordenar las cajas del examen 9

9. Provincias y localidades en un componente 10

10. Ingresos y egresos con máximo de 120 11

11. Respaldo CP y hook useLocalidades 12

12. Respaldo filtrar personas con un efecto 13

13. Cómo resolver los huecos sin adivinar 14

14. Ruta de estudio y criterios de práctica 15

15. Referencias y alcance 16

2. Estado y manejo de variables

Qué significa cada parte

CÓDIGO · JSX
```jsx
import { useState } from "react";

const [buscar, setBuscar] = useState("");
```

`buscar` es el valor del render actual; `setBuscar` solicita cambiarlo; `""` es el valor inicial. La declaración se escribe dentro de la función del componente, antes del return. Importar el hook no crea ningún estado. [R1]

Necesidad	Inicialización adecuada
Texto del input	`useState("")`
Cantidad de personas	`useState(0)`
Lista de localidades	`useState([])`
Indicador de carga	`useState(false)`
Varios campos de persona	`useState({ nombre: "", apellido: "" })`

Cómo se actualiza

CÓDIGO · JSX
```jsx
setBuscar("Rosario");
setCantidad(anterior => anterior + 1);
```

Para un valor que depende del anterior, la forma funcional evita usar una captura vieja. No escribas `buscar = "Rosario"` ni incrementes directamente el estado. Los hooks van en el nivel superior del componente, fuera de condiciones, bucles y manejadores. [R1]

El estado es una foto del render actual

CÓDIGO · JSX
```jsx
const [cantidad, setCantidad] = useState(0);

function ingresar() {
  setCantidad(cantidad + 1);
  console.log(cantidad); // En el primer clic: 0.
}
```

El setter no cambia la variable que está leyendo ese manejador. En el siguiente render, cantidad vale 1. Una variable local como `let anonimas = 0` se inicializa otra vez cada vez que se ejecuta el componente; modificarla no solicita un render. [R2]

3. Input controlado y eventos

El patrón que tenés que reconocer y completar

CÓDIGO · JSX
```jsx
const [buscar, setBuscar] = useState("");

function handleChange(e) {
  setBuscar(e.target.value);
}

// Dentro del return:
<input type="text" value={buscar} onChange={handleChange} />
```

Un input controlado muestra el estado mediante `value`. Cuando el usuario escribe, `onChange` llama al manejador; este lee `e.target.value` y usa el setter. React vuelve a renderizar y entrega el valor actualizado al input. Si hay value pero no una actualización del estado, no se puede editar normalmente. [R3]

Expresión	Qué representa
`e`	Objeto del evento recibido por el manejador.
`e.target`	Elemento que originó el evento, en estos ejemplos el input.
`e.target.value`	Valor actual del input; en un input de texto es un string.
`e.target.name`	Nombre del campo, útil con varios inputs.
`value={buscar}`	El valor que React muestra; no es el lugar que lo almacena.

Cuatro expresiones parecidas con resultados distintos

CÓDIGO · JSX
```jsx
onChange={handleChange}                  // Pasa la función.
onChange={(e) => setBuscar(e.target.value)} // También funciona.
onChange={setBuscar(e.target.value)}     // Mal: ejecuta al renderizar.
onChange={() => handleChange}           // Mal: no la invoca.
```

La última función devuelve la referencia a handleChange y no procesa el evento. Si querés una envoltura, usá `onChange={(e) => handleChange(e)}`. Para un botón sin argumentos: `onClick={fncIngreso}` o `onClick={() => fncIngreso()}`. [R4]

Error de una letra de atraso

CÓDIGO · JSX
```jsx
function handleChange(e) {
  const texto = e.target.value;
  setBuscar(texto);
  console.log(texto); // Texto nuevo; buscar todavía es el anterior.
}
```

Si dentro de este mismo manejador filtrás o buscás, usá texto o e.target.value. Si lo hacés en un efecto dependiente de buscar, el efecto usará el valor del nuevo render. [R2]

4. Búsqueda con al menos 3 caracteres

La frase del enunciado significa que 3 ya debe estar permitido. El nombre correcto de la propiedad es `length`, en minúscula y sin paréntesis. No es `Length`, `Leghnt` ni `length()`. [R5]

Texto	Longitud	¿Puede buscar?	disabled con longitud menor a 3
""	0	No	true
"ab"	2	No	true
"abc"	3	Sí	false
"abcd"	4	Sí	false

Versión completa y coherente con el formulario original

CÓDIGO · JSX

```jsx
import { useState } from "react";

export default function FormularioUsuario() {
  const [buscar, setBuscar] = useState("");

  const manejarEnvio = (e) => {
    e.preventDefault();
    if (buscar.length < 3) return;
    console.log("Buscar con:", buscar);
  };

  return (
    <form onSubmit={manejarEnvio}>
      <label>Buscar:</label>
      <input type="text" value={buscar}
        onChange={(e) => setBuscar(e.target.value)} />
      <button type="submit" disabled={buscar.length < 3}>
        Enviar
      </button>
    </form>
  );
}
```

Las piezas centrales son la declaración del estado, onChange y la condición de longitud. `disabled={true}` deshabilita; por eso se pone la condición de invalidez. Con `> 3` excluirías el caso de exactamente 3 si lo usaras para habilitar. [R6]

La guardia `if (...) return` impide ejecutar la búsqueda inválida dentro del manejador; disabled comunica y bloquea la acción desde el botón. `preventDefault()` solamente evita el envío nativo del form: no valida ni habilita nada. Se incluye para que este ejemplo con form no recargue la página; si la consigna pide sólo bloquear el botón, la línea central es disabled. [R4]

5. Condiciones y símbolos que pueden aparecer

Separá el evento de la condición

CÓDIGO · JSX
```
<input value={buscar}
  onChange={(e) => setBuscar(e.target.value)} />
<button disabled={buscar.length < 3}>Buscar</button>
```

El input cambia el estado; el botón consulta ese estado. No se escribe `onChange{disabled=...}`. Si el botón está dentro de un form y debe manejar un clic sin envío nativo, usá type="button" y onClick; en ese caso no necesitás un manejador onSubmit.

CÓDIGO · JSX
```
<button type="button" disabled={buscar.length < 3}
  onClick={() => console.log("Buscar con:", buscar)}>
  Buscar
</button>
```

Operador ternario: una de dos opciones

CÓDIGO · JSX
```
const puedeBuscar = buscar.length >= 3;
const mensaje = puedeBuscar ? "Listo" : "Faltan caracteres";
// Dentro del return:
<button disabled={puedeBuscar ? false : true}>Buscar</button>
```

Se lee: condición ? valor si es verdadera : valor si es falsa. En disabled, el ternario anterior se simplifica a `disabled={!puedeBuscar}`. El signo `!` niega un booleano. No escribas solamente `buscar > 3`: estás comparando un texto con un número, no midiendo su longitud. [R7]

Doble ampersand: &&

CÓDIGO · JSX
```
{buscar.length < 3 && <span>Ingresá al menos 3 caracteres</span>}
{error && <span style={{ color: "red" }}>{error}</span>}
```

Cuando la condición es verdadera, se evalúa y muestra el elemento de la derecha. Si es false, React no muestra ese elemento. En lógica, `a && b` exige ambas condiciones; no es lo mismo que `&`. Evitá `cantidad && <p>...</p>` si cantidad puede ser 0: React puede mostrar el 0. Escribí `cantidad > 0 && ...`. [R8]

Comparadores indispensables

=== compara igualdad; = asigna. < significa menor, <= menor o igual; > mayor, >= mayor o igual. Para un mínimo de 3: permitir con >= 3 o impedir con < 3. Para un máximo de 120: impedir nuevos ingresos cuando la cantidad ya es >= 120.

6. Spread y handleChange con objetos

El estado puede guardar varios campos juntos

CÓDIGO · JSX

```jsx
const [persona, setPersona] = useState({
  apellido: "", nombre: "", cp: "", localidad: ""
});

function handleChange(e) {
  const { name, value } = e.target;
  setPersona(anterior => ({ ...anterior, [name]: value }));
}

// Dentro del return:
<input name="apellido" value={persona.apellido}
  onChange={handleChange} />
<input name="nombre" value={persona.nombre}
  onChange={handleChange} />
```

El nombre handleChange es una convención: también podrías llamarla manejarCambio. Lo importante es conectar la misma función y usar un name que coincida con la propiedad que debe cambiar. [R9]

Lectura de la línea central

- `const { name, value } = e.target` extrae dos propiedades del input.
- `anterior` representa el objeto de estado previo.
- `...anterior` copia sus propiedades en un objeto nuevo; es spread.
- `[name]: value` usa el contenido de name como nombre de propiedad.
- Las propiedades posteriores reemplazan a las anteriores si tienen la misma clave.
- `({ ... })` devuelve un objeto desde la función flecha. Sin los paréntesis externos, las llaves pueden interpretarse como el cuerpo de la función. [R9, R10]

Si el input tiene name="apellido" y se escribe Pérez, cambia apellido y conserva nombre, cp y localidad. `[name]` no crea un arreglo en esta posición: es una clave calculada del objeto.

Los errores que borran o no actualizan datos

CÓDIGO · JSX

```jsx
setPersona({ apellido: e.target.value }); // Pierde los otros campos.
persona.apellido = e.target.value;        // Muta el objeto del estado.
```

El setter reemplaza el objeto; no fusiona sus propiedades por vos. Una forma válida para un solo campo es `setPersona(anterior => ({ ...anterior, apellido: valor }))`. Spread crea una copia superficial: alcanza para este objeto de campos simples. [R9, R10]

7. useEffect y sus dependencias

useEffect sirve para sincronizar con algo externo, por ejemplo cargar datos de una API. Se declara dentro del componente, antes del return. El callback se ejecuta después del commit; no hace falta poner las consultas dentro del JSX. [R11]

Forma	Cuándo se ejecuta
`useEffect(() => { ... })`	Después de cada commit del componente.
`useEffect(() => { ... }, [])`	Al montar; no se repite por simples cambios de estado.
`useEffect(() => { ... }, [provincia])`	Al montar y cuando cambia provincia.
`useEffect(() => { ... }, [buscar, personas])`	Al montar y cuando cambia cualquiera de esos valores.

En desarrollo, Strict Mode puede ejecutar un ciclo extra de preparación y limpieza. Para resolver estos modelos, usá la regla de montaje y dependencias; no confundas ese chequeo con una recarga de página. [R11]

Cargar datos y saber qué se guarda

CÓDIGO · JSX

```jsx
useEffect(() => {
  async function cargar() {
    const response = await fetch("/api/provincias");
    const data = await response.json();
    setProvincias(data);
  }
  cargar();
}, []);
```

Esta plantilla supone una API que devuelve directamente un arreglo. Si devuelve `{ results: [...] }`, se guarda data.results. El resultado depende del contrato de la API; no se agrega results por costumbre. La dirección es ilustrativa. [R11]

Cargar según la selección

En un efecto con `[provincia]`, una guardia `if (provincia === "") return;` evita consultar sin selección durante la ejecución inicial. Las dependencias se eligen por los valores reactivos usados, no para forzar una cantidad deseada de ejecuciones. El capítulo 9 lo muestra completo. [R12]

Cómo detectar un ciclo

Sin segundo argumento, una consulta que guarda un nuevo arreglo puede causar: petición, setter, render, commit, nuevo efecto y otra petición. También puede ocurrir si un efecto depende de localidades y cada ejecución reemplaza localidades con otro arreglo. No todo efecto sin segundo argumento genera un bucle: necesita provocar actualizaciones que lo vuelvan a ejecutar. [R12]

8. Ordenar las cajas del examen

La secuencia que informó el profesor es **11 → 12 → 2 → 1 → 6 → 5**. Aprendé la función de cada bloque para reconocer variantes, no solamente sus números.

Caja	Intención del bloque	Por qué va allí
11	Importar useEffect y useState	Permite utilizar ambos hooks.
12	Declarar el estado provincias como arreglo	Es la lista de origen.
2	Declarar el estado localidades como arreglo	Es la lista que se quiere recuperar y mostrar.
1	Cargar provincias con un efecto y []	Es la carga inicial.
6	Recuperar localidades según una provincia	Depende de la selección o del dato de provincia utilizado.
5	Mapear localidades dentro del JSX	Muestra el resultado.

Corrección a tu interpretación

En esa secuencia, primero aparece el estado de provincias, caja 12, y después el de localidades, caja 2. Ambos estados van antes de los efectos. React no exige que esos dos estados se declaren en ese orden específico; lo importante es mantener llamadas a hooks incondicionales y un orden estable entre renders. Para este modelo, conservá la secuencia del docente.

Por qué se omiten las otras cajas

- 3 y 8: en la transcripción importan solamente useEffect; no cubren ambos hooks y además contienen signos incorrectos.
- 4: no representa la carga inicial correcta de provincias y usa localidades como dependencia.
- 7: busca localidades dependiendo de localidades, en vez de la selección; puede realimentar las consultas.
- 9: inicia localidades como texto y escribe useState incorrectamente; para mapear una lista se espera un arreglo.
- 10: muestra provincias. Puede servir para otra parte de una interfaz, pero no reemplaza el mapa de localidades pedido.

Una inconsistencia de la caja 6

Tu transcripción usa provincia en la URL y `[provincias]` en las dependencias. Lista de provincias y provincia seleccionada son datos distintos. Si el efecto usa una variable seleccionada llamada provincia, debe incluir `[provincia]`. Si depende solamente de la lista, no se ejecutará por cambiar una selección separada. El profesor eligió la caja 6 por su intención; no memorices el plural como una regla técnica.

El mapa mental es: importaciones, función del componente, estados, manejadores y efectos, return con la lista. Los manejadores pueden escribirse antes o después de los efectos; no hay una única posición obligatoria para ellos. [R1, R11, R13]

9. Provincias y localidades en un componente

Ejemplo de estructura con selección explícita. Las rutas son ilustrativas: se supone que ambas API devuelven arreglos de objetos `{ id, nombre }`. No requiere Context. Para estudiar el orden, las consultas están abreviadas con promesas; los métodos de fetch equivalen a esperar la respuesta y leer su JSON.

CÓDIGO · JSX

```jsx
import { useEffect, useState } from "react";

export default function Selector() {
  const [provincias, setProvincias] = useState([]);
  const [localidades, setLocalidades] = useState([]);
  const [provincia, setProvincia] = useState("");

  useEffect(() => {
    fetch("/api/provincias")
      .then(r => r.json())
      .then(datos => setProvincias(datos));
  }, []);

  useEffect(() => {
    setLocalidades([]);
    if (provincia === "") return;
    fetch(`/api/localidades/${provincia}`)
      .then(r => r.json())
      .then(datos => setLocalidades(datos));
  }, [provincia]);

  return (
    <div>
      <select value={provincia}
        onChange={e => setProvincia(e.target.value)}>
        <option value="">Elegí una provincia</option>
        {provincias.map(p => (
          <option key={p.id} value={p.id}>{p.nombre}</option>
        ))}
      </select>
      <ul>
        {localidades.map(l => <li key={l.id}>{l.nombre}</li>)}
      </ul>
    </div>
  );
}
```

Provincias guarda las opciones; provincia guarda el id seleccionado; localidades guarda el resultado de la segunda consulta. El input select cambia la selección y eso activa su efecto. Las comillas invertidas permiten insertar `${provincia}` en la URL. Map transforma cada objeto en JSX y key identifica al elemento. [R13]

Es una plantilla de examen: omite manejo de errores y cancelación de peticiones. En una app real hay que contemplar fallos y respuestas que lleguen fuera de orden si el usuario cambia rápidamente de provincia. No necesitás agregar esa complejidad para aprender estos bloques.

10. Ingresos y egresos con máximo de 120

El dato que cambia es la cantidad presente. Se inicia en 0, suma al ingresar, resta al egresar y se verifica el límite antes de sumar. Este ejemplo avisa cuando se intenta ingresar estando ya completo.

CÓDIGO · JSX

```jsx
import { useState } from "react";

export default function IngresoLocal() {
  const [cantidad, setCantidad] = useState(0);

  function fncIngreso() {
    if (cantidad >= 120) {
      alert("Límite de 120 personas alcanzado");
      return;
    }
    setCantidad(anterior => anterior + 1);
  }

  function fncEgreso() {
    if (cantidad <= 0) return;
    setCantidad(anterior => anterior - 1);
  }

  return (
    <div>
      <h4>Ingresos Local</h4>
      <p>Cantidad de Personas: {cantidad}</p>
      <button onClick={fncIngreso}>Ingreso</button>
      <button onClick={fncEgreso} disabled={cantidad === 0}>
        Egreso
      </button>
    </div>
  );
}
```

Comprobación de los límites

- Con 119, se permite ingresar y se llega a 120.
- Con 120, se muestra la alerta y return impide llegar a 121.
- Con 0, no se permite egresar y nunca se llega a -1.

También podés bloquear Ingreso con `disabled={cantidad >= 120}`. Pero un botón deshabilitado no ejecuta su onClick: si el requisito incluye alerta, deberás mostrarla al alcanzar el límite o indicar el estado completo de otra manera. En la versión anterior se deja el botón activo para avisar al intentar excederlo.

Si la consigna exige avisar exactamente al pasar de 119 a 120, calculá el próximo valor dentro del manejador, actualizá el estado y alertá si ese próximo valor es 120. No pongas alert dentro de la función actualizadora del setter: esa función debe ser pura. [R1]

11. Respaldo CP y hook useLocalidades

Este modelo combina un objeto persona, un manejador compartido y una consulta por código postal. Context queda encapsulado en el hook que te proveen. No hace falta construirlo para completar handleChange.

Qué se puede saber y qué no

CÓDIGO · JSX
```jsx
const [persona, setPersona] = useState({
  apellido: "", nombre: "", cp: "", localidad: ""
});
const { localidades, devLocalidadPorCP } = useLocalidades();
```

La transcripción no define qué devuelve devLocalidadPorCP ni si es asíncrona. Por eso no hay una única implementación demostrable. El siguiente ejemplo supone que recibe un CP, devuelve inmediatamente el nombre de una localidad y devuelve undefined si no lo encuentra. Es un contrato ilustrativo, no una afirmación sobre el hook del profesor.

CÓDIGO · JSX
```jsx
function handleChange(e) {
  const { name, value } = e.target;

  if (name === "cp") {
    const localidad = devLocalidadPorCP(value);
    setPersona(anterior => ({
      ...anterior,
      cp: value,
      localidad: localidad ?? ""
    }));
  } else {
    setPersona(anterior => ({ ...anterior, [name]: value }));
  }
}
```

Si cambia apellido o nombre, se actualiza ese campo. Si cambia cp, se busca con el valor nuevo y se guardan cp y localidad juntos. `?? ""` usa texto vacío cuando el resultado es null o undefined. Evitá buscar con persona.cp inmediatamente después del setter: todavía representa el render anterior. [R2, R9]

Cómo adaptarlo a lo que realmente te den

- Si devuelve un objeto con nombre, obtené su propiedad, por ejemplo resultado.nombre; no guardes todo el objeto para mostrarlo como texto.
- Si devuelve una promesa, hay que esperar su resultado con await o then; no guardar la promesa en persona.localidad.
- Si solamente actualiza localidades dentro del hook y no devuelve una localidad, se consume ese estado; no se inventa un resultado de retorno.
- Si el JSX muestra error, esa variable debe venir del hook o declararse, por ejemplo `const [error, setError] = useState("")`. El mensaje con && no la crea.

El patrón que sí tenés que dominar es: identificar name, leer value, conservar el objeto y actualizar el campo correcto. No dediques primero el tiempo a implementar Context ni a adivinar su API.

12. Respaldo filtrar personas con un efecto

El modelo antiguo guarda personas como lista original, buscar como texto y personasFiltradas como resultado. Filter conserva los elementos que cumplen la condición; startsWith comprueba el comienzo del texto y toLowerCase permite ignorar mayúsculas. [R13, R14]

Versión con el bloque agregado

CÓDIGO · JSX

```jsx
const [personas, setPersonas] = useState([]);
const [personasFiltradas, setPersonasFiltradas] = useState([]);
const [buscar, setBuscar] = useState("");

useEffect(() => {
  setPersonas(["Alberto", "Alejandro", "Juan", "Maria", "Pedro", "Ana"]);
}, []);

function handleChange(e) {
  setBuscar(e.target.value);
}

useEffect(() => {
  setPersonasFiltradas(personas.filter(persona =>
    persona.toLowerCase().startsWith(buscar.toLowerCase())
  ));
}, [buscar, personas]);
```

Este fragmento va dentro de BuscarPersona, antes de su return. El efecto no puede quedar después de la llave que cierra el componente. Se conservan el input controlado y el map de personasFiltradas dentro del return.

Las tres respuestas del modelo

1. El bloque agregado recalcula personasFiltradas cuando cambia buscar o personas, y también tiene ejecución inicial. Con buscar igual a "al", obtiene Alberto y Alejandro.
2. Hay que quitar de handleChange la actualización de personasFiltradas: esa responsabilidad pasa al efecto. El manejador solamente cambia buscar. También hay que ubicar el efecto dentro del componente.
3. Personas conserva el origen completo; personasFiltradas guarda una vista para mostrar sin destruir el origen. Si filtrás reemplazando la lista original, podés perder candidatos al borrar o cambiar la búsqueda.

Con buscar vacío, startsWith("") es verdadero para todos esos nombres: la versión con efecto termina mostrando toda la lista. Si se exige un mínimo de 3 caracteres, agregá esa condición y guardá [] mientras no se cumpla. [R14]

Alternativa más simple si no te exigen ese efecto

CÓDIGO · JSX

```jsx
const personasFiltradas = personas.filter(persona =>
  persona.toLowerCase().startsWith(buscar.toLowerCase())
);
```

Esto se calcula durante el render sin un segundo estado. Para analizar el examen explicá su versión; para una solución libre podés evitar estado derivado redundante. No mezcles esta constante con otro useState del mismo nombre. [R15]

13. Cómo resolver los huecos sin adivinar

Primero seguí el dato

1. ¿Qué cambia? Texto: string; cantidad: número; resultados: arreglo; formulario: objeto.
2. ¿Qué acción lo cambia? Escritura: onChange; clic: onClick; envío: onSubmit; cambio de selección que carga datos: useEffect con dependencia.
3. ¿De dónde sale el valor nuevo? Del evento, del estado anterior o de la respuesta de una consulta.
4. ¿Hay una condición? Escribila con el límite exacto y probá el caso justo antes, justo en el límite y justo después.
5. ¿Dónde se ve? value, texto entre llaves o map dentro del return.

Recetas de una a tres líneas

CÓDIGO · JSX
```
const [buscar, setBuscar] = useState("");
onChange={e => setBuscar(e.target.value)}
disabled={buscar.length < 3}
```

CÓDIGO · JSX
```
if (buscar.length < 3) return;
if (cantidad >= 120) return;
setCantidad(anterior => anterior + 1);
```

CÓDIGO · JSX
```
const { name, value } = e.target;
setPersona(anterior => ({ ...anterior, [name]: value }));
```

Las primeras recetas mezclan declaraciones y atributos: no se pegan juntas en cualquier lugar. La declaración va en el cuerpo del componente, los atributos en la etiqueta JSX y las guardias en el manejador. Si ya existe un callback o un estado, completalo; no lo declares dos veces.

Detalles pequeños frente a errores que cambian el programa

Un punto y coma omitido muchas veces es aceptable en JavaScript. En cambio, no invocar el callback, invertir < por >, usar la dependencia equivocada o perder propiedades del objeto cambia el comportamiento. React distingue onChange de onchange, useState de usestate y setLocalidades de setlocalidades: los nombres deben coincidir.

El input se cierra con `/>`; onSubmit y onClick reciben expresiones con `={...}`; la función exportada se declara `export default function Nombre() { ... }`. No uses `function export default` ni dejes el JSX fuera del return. Para agrupar varios elementos usá un contenedor o fragmento.

Si vuelve el contador del primer parcial

Con una variable local inicializada en 0 y un estado inicializado en 0: primer clic, pantalla 0 y 1; consola local 1 y estado 0. Segundo clic, tras completarse el render anterior: pantalla 0 y 2. Para igualarlas, reemplazá la variable local por otro useState y su incremento por el setter correspondiente. No dejes ambas declaraciones con el mismo nombre.

14. Ruta de estudio y criterios de práctica

Primera vuelta antes del recuperatorio

Bloque	Tiempo orientativo	Acción concreta
Estado y eventos, capítulos 2 y 3	20 minutos	Explicá cada nombre y reproducí el patrón del input.
Búsqueda y condiciones, 4 y 5	25 minutos	Escribí estado, onChange, disabled y guardia; probá 2, 3 y 4 caracteres.
Objetos y spread, 6	15 minutos	Actualizá apellido conservando los demás campos.
Efectos y orden, 7 a 9	25 minutos	Reconstruí las cajas y distinguí lista de selección.
Límite de personas, 10	15 minutos	Comprobá 0, 119, 120 y el intento de 121.
Respaldo, 11 y 12	20 minutos	Entendé los supuestos del hook y respondé las tres preguntas del filtro.

Son unas dos horas para una primera vuelta activa, no una promesa de dominio ni el tiempo total que necesitarás. Si tenés menos margen, empezá por input, longitud y cajas; después hacé spread y capacidad. Reservá la siguiente etapa para practicar sin mirar soluciones.

Comprobación final antes de pasar a simulacros

- Puedo explicar por qué disabled lleva < 3 y no > 3.
- Puedo conectar un input al setter sin ejecutar la función durante el render.
- Puedo actualizar un campo sin borrar los otros.
- Puedo explicar por qué localidades depende de la provincia elegida.
- Puedo ordenar los seis bloques del modelo y describir cada uno.
- Puedo sumar y restar sin superar 120 ni bajar de 0.

Cómo vamos a corregir cuando me avises que terminaste

Cada simulacro tendrá 2 o 3 ejercicios breves para completar o corregir código, uno de ordenar y alguna pregunta de interpretación. No empezamos esa etapa hasta que lo indiques.

Cada respuesta será Bien, Medio o Mal. Si comprendiste el núcleo y resolviste el comportamiento, reconoceré ese mérito; una errata menor aislada no anulará todo el ejercicio. Si falta una pieza, marcaré cuál y explicaré la corrección. Los errores de límite, evento, estado o dependencia se evaluarán por su impacto.

La nota orientativa será de 1 a 8, con máximo 8. Según lo que informó tu profesor, la referencia para promocionar es 7. No conocemos su rúbrica exacta: nuestras notas servirán para detectar qué falta dominar, no para asegurar una calificación del recuperatorio.

15. Referencias y alcance

Esta guía reconstruye los ejercicios a partir de los enunciados y aclaraciones que compartiste. Los números de cajas y las prioridades proceden de ese relato. Los ejemplos corrigen la sintaxis e indican los supuestos cuando falta el contrato de una API o de un hook.

Documentación para los conceptos usados

- [R1] React useState:
  https://react.dev/reference/react/useState
- [R2] React State as a Snapshot:
  https://react.dev/learn/state-as-a-snapshot
- [R3] React input:
  https://react.dev/reference/react-dom/components/input
- [R4] React Responding to Events:
  https://react.dev/learn/responding-to-events
- [R5] MDN String length:
  https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/length
- [R6] MDN disabled:
  https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Attributes/disabled
- [R7] MDN operador ternario:
  https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Conditional_operator
- [R8] React Conditional Rendering:
  https://react.dev/learn/conditional-rendering
- [R9] React Updating Objects in State:
  https://react.dev/learn/updating-objects-in-state
- [R10] MDN Spread syntax:
  https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Spread_syntax
- [R11] React useEffect:
  https://react.dev/reference/react/useEffect
- [R12] React Synchronizing with Effects:
  https://react.dev/learn/synchronizing-with-effects
- [R13] React Rendering Lists:
  https://react.dev/learn/rendering-lists
- [R14] MDN startsWith:
  https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/startsWith
- [R15] React You Might Not Need an Effect:
  https://react.dev/learn/you-might-not-need-an-effect

No se agregan como unidades nuevas Router, Zod avanzado, arquitectura de Context ni herramientas posteriores al parcial. Para impedir una búsqueda corta alcanza una condición aplicada al estado: escribir el nombre de una biblioteca no ejecuta una validación.